Universidad Autónoma del Estado de Hidalgo

UNIVERSIDAD AUTÓNOMA DEL ESTADO DE HIDALGO

ESCUELA SUPERIOR DE TLAHUELILPAN

Reporte de Práctica:

Construcción de Analizador Sintáctico y Generación del AST

INTEGRANTES:
CORONA CORTÉS ERNESTO ANTONIO
FLORES CORDERO JONATHAN
RAMIREZ CERRITO ALAN

GRUPO: 301

1.Introducción

El desarrollo de compiladores y traductores de lenguaje representa uno de los pilares fundamentales de la ciencia de la computación, al permitir transformar especificaciones formales en representaciones ejecutables o estructuras intermedias procesables. En la fase inicial de este proceso —el procesamiento frontal o *front-end*—, el análisis léxico y el análisis sintáctico trabajan de forma coordinada para validar la estructura del código fuente frente a la gramática definida.

El presente reporte documenta el diseño, la implementación y la verificación de un analizador sintáctico con capacidad de generación de Árboles Sintácticos Abstractos (AST) enfocado en expresiones aritméticas y asignaciones. Para automatizar la construcción de los analizadores, se utilizó Flex (calc-lexico.l) para el reconocimiento y tokenización de caracteres, y Bison (calc-sintaxis.y) para la definición BNF de la gramática libre de contexto y el manejo de precedencia de operadores. Adicionalmente, el sistema construye en memoria una representación en C (ast.h y ast.c) que se exporta en formato DOT para su posterior renderización gráfica mediante Graphviz (ast_tree.png).

El proyecto se desarrolló bajo una metodología de ingeniería de software colaborativa utilizando Git con una estrategia de ramificación , asegurando la modularidad del código, el control de versiones incremental y una distribución equitativa de responsabilidades entre los tres integrantes del equipo.

2.Objetivos

Objetivo General

Desarrollar un analizador sintáctico para expresiones aritméticas utilizando Flex y Bison, integrando la generación e inspección visual del Árbol Sintáctico Abstracto (AST) mediante Graphviz.

Objetivos Específicos

1. Definir la especificación de tokens y expresiones regulares en calc-lexico.l.
2. Implementar la gramática BNF y la precedencia de operadores en calc-sintaxis.y.
3. Diseñar las estructuras de datos en C (ast.h y ast.c) para construir el AST en memoria y exportarlo a formato DOT.
4. Utilizar un flujo de trabajo distribuido en Git basado en ramas y Pull Requests

3.Desarrollo e Implementación

A. Arquitectura del Proyecto y División de Trabajo

Archivo	Responsable	Descripción / Función
calc-lexico.l	Jonathan	Tokenizador (Flex). Reconoce números, operadores (+, -, *, /, =) e identificadores.
calc-sintaxis.y	Ernesto	Gramática independiente del contexto (Bison). Muestra reglas de producción y manejo de precedencia.
ast.h / ast.c	Alan	Define nodos del AST y la función recursiva que genera el archivo ast_tree.dot.
input.txt	Ernesto	Cadena de prueba con expresiones aritméticas válidas.

B. Flujo de Compilación y Ejecución

Describe la canalización (*pipeline*) empleada para traducir las definiciones formales a código máquina y gráficos:

1. **Análisis Léxico:** Generación de lex.yy.c con flex calc-lexico.l.
2. **Análisis Sintáctico:** Generación de calc-sintaxis.tab.c y cabeceras con bison -d calc-sintaxis.y.
3. **Enlace y Compilación:** Generación del ejecutable mediante gcc lex.yy.c calc-sintaxis.tab.c ast.c -o calc.exe.
4. **Ejecución:** Redirección de entrada ./calc.exe < input.txt para producir ast_tree.dot.
5. **Renderizado de la Imagen:** Conversión gráfica con dot -Tpng ast_tree.dot -o ast_tree.png.

4. Pruebas y Resultados

Proceso de Compilación

```
ernes@chino28 ~
$ cd "C:/Users/ernes/CLionProjects/Equipo3"

ernes@chino28 /cygdrive/c/Users/ernes/CLionProjects/Equipo3
$ ls
CMakeLists.txt     analizador.y          calc-sintaxis.y     lexer.l
README.md          analizadorLexico.py   cmake-build-debug   lexer2.l
a.exe              ast.c                 ejemplo.txt         out.txt
analizador.tab.c   ast.h                 input.txt           programa.txt
analizador.tab.h   calc-lexico.l         lex.yy.c            programac.txt

ernes@chino28 /cygdrive/c/Users/ernes/CLionProjects/Equipo3
$ flex calc-lexico.l

ernes@chino28 /cygdrive/c/Users/ernes/CLionProjects/Equipo3
$ bison -d calc-sintaxis.y

ernes@chino28 /cygdrive/c/Users/ernes/CLionProjects/Equipo3
$ gcc lex.yy.c calc-sintaxis.tab.c ast.c -o calc.exe
calc-lexico.l:3:13: warning: "_POSIX_C_SOURCE" redefined
    3 |      #define _POSIX_C_SOURCE 200809L
      |
In file included from /usr/include/sys/config.h:5,
                 from /usr/include/_ansi.h:11,
                 from /usr/include/stdio.h:29,
                 from lex.yy.c:18:
/usr/include/sys/features.h:154:9: note: this is the location of the previous definition
  154 | #define _POSIX_C_SOURCE          202405L
      |

ernes@chino28 /cygdrive/c/Users/ernes/CLionProjects/Equipo3
$ dot -Tpng ast_tree.dot -o ast_tree.png
Error: dot: can't open ast_tree.dot: No such file or directory

ernes@chino28 /cygdrive/c/Users/ernes/CLionProjects/Equipo3
$ ./calc.exe < input.txt

----------------------------------------
Seudo-assembly escrito en 'programa.sasm'
----------------------------------------
Resultado (eval_nodo): 23

 EJECUCION DEL PROGRAMA

Arbol Sintáctico Abstracto (AST):
----------------------------------------
SEQ
    DECL       a
        INT        5
    SEQ
        DECL       b
        SEQ
            ASSIGN     b
                INT        6
            SEQ
                DECL       sum
```

```
Seudo-assembly escrito en 'programa.sasm'
----------------------------------------
Resultado (eval_nodo): 23

 EJECUCION DEL PROGRAMA

Arbol Sintáctico Abstracto (AST):
----------------------------------------
SEQ
    DECL       a
        INT        5
    SEQ
        DECL       b
        SEQ
            ASSIGN     b
                INT        6
            SEQ
                DECL       sum
                    SUMA
                        ID         a
                        MULT
                            ID         b
                            INT        3
                SEQ
                    RETURN
                        ID         sum

Tabla de Símbolos:
----------------------------------------
Variable Valor       Estado
----------------------------------------
a        5           Inicializado
b        6           Inicializado
sum      23          Inicializado

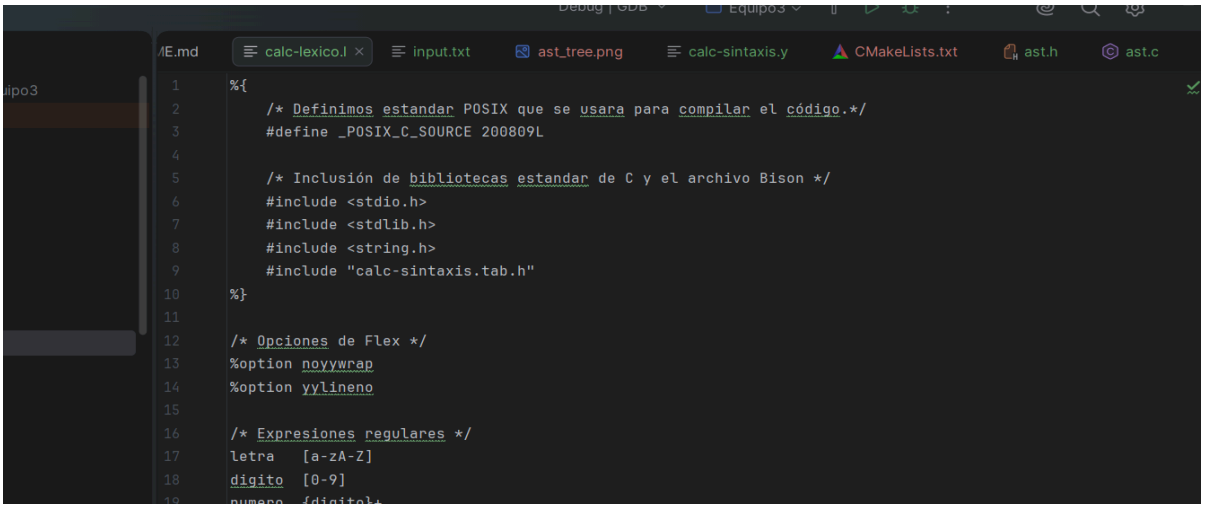
 FIN DE LA EJECUCION

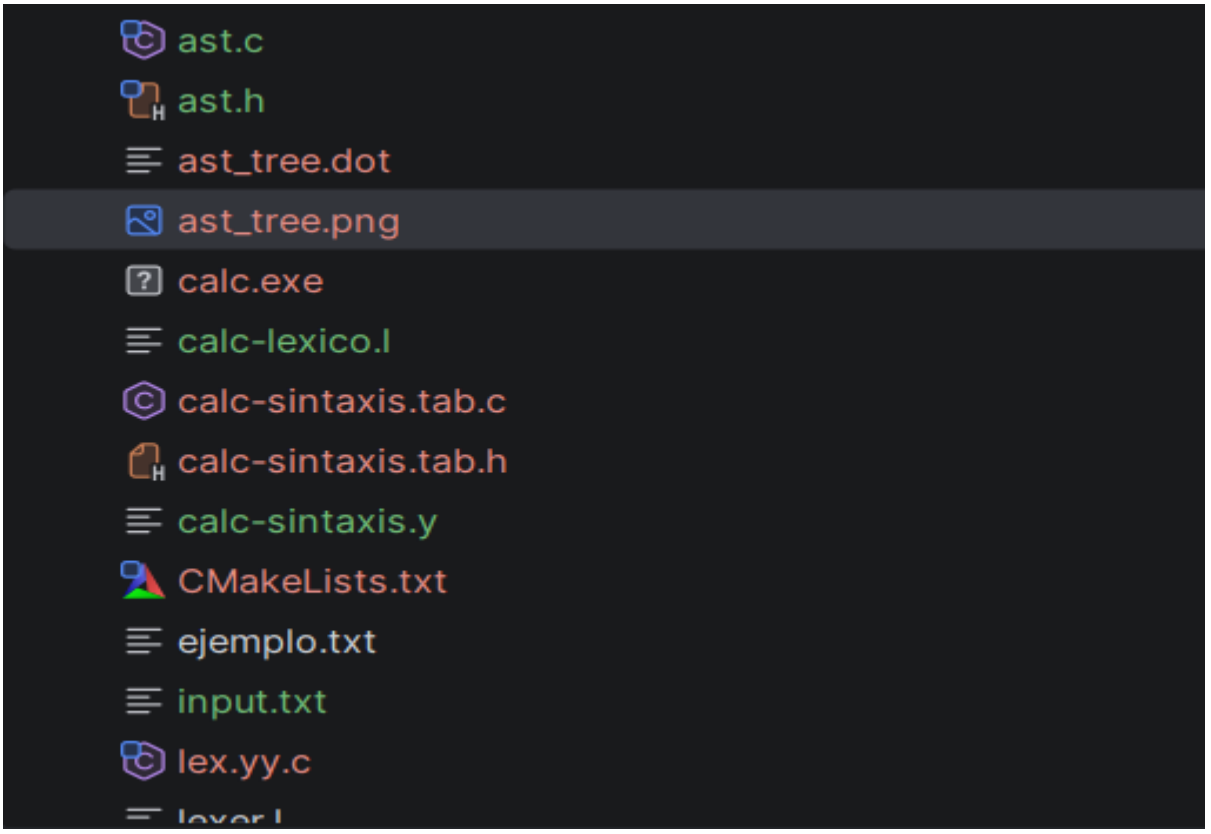
 Resultado del Programa

    23
    ✔ No hay errores

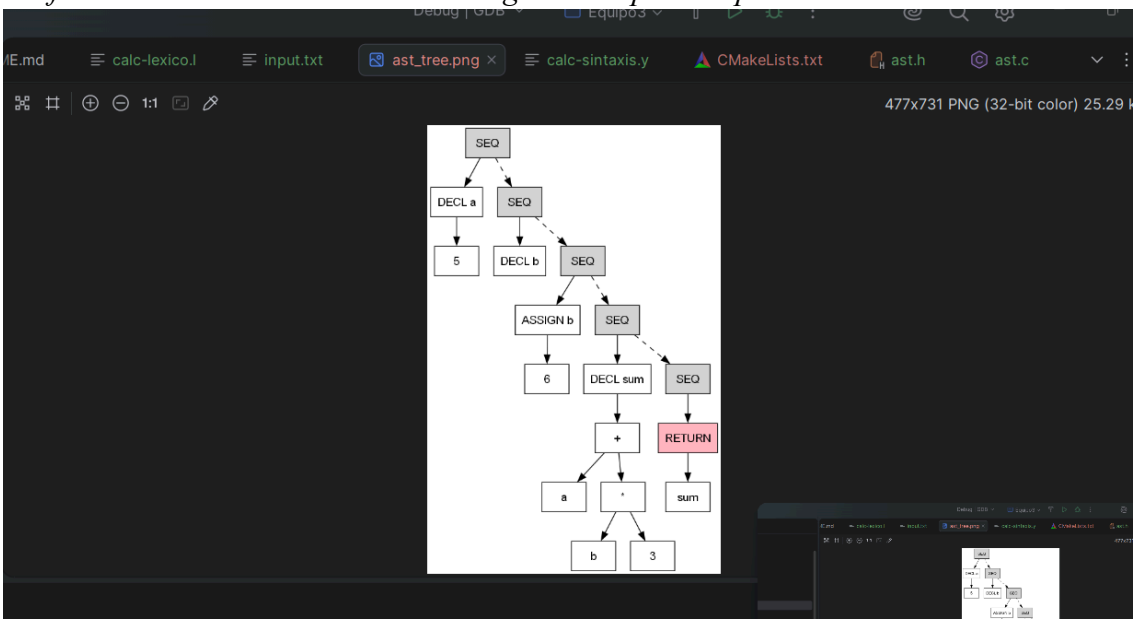
ernes@chino28 /cygdrive/c/Users/ernes/CLionProjects/Equipo3
$ dot -Tpng ast_tree.dot -o ast_tree.png
```

Archivos cargados y generados





Gráfica del Árbol Sintáctico Abstracto generado por Graphviz.



5.Conclusión

En esta práctica logramos construir un analizador sintáctico funcional en C utilizando Flex y Bison. La combinación de ambas herramientas facilitó dividir el proceso en dos partes claras: el reconocimiento de tokens y la aplicación de las reglas gramaticales, asegurando que las operaciones matemáticas se evalúen con la precedencia correcta.

Por otro lado, la construcción del Árbol Sintáctico Abstracto (AST) y su exportación a imagen con Graphviz (ast_tree.png) fue clave para comprobar visualmente que interpreta la estructura de las expresiones tal como se esperaba. Finalmente, el trabajo colaborativo mediante ramas en Git nos permitió organizarnos eficientemente entre los tres integrantes del equipo, registrando los avances de manera ordenada y sin sobrescribir el código de los demás.